SKILL-3

ID:2400030211

Name: RACHAKONDA HEMALATHA

Section:59

#SKILL - 3 (Working with HQL - Sorting, Pagination & Aggregates)

Prerequisites:

- Basic knowledge of Hibernate
- Basic idea of HQL syntax

In this lab, you will extend the Hibernate module created in Skill 2 and perform advanced data retrieval using HQL, including sorting, pagination, filtering, and aggregate functions. These operations help in retrieving structured and meaningful data from the database using Hibernate.

Tasks:

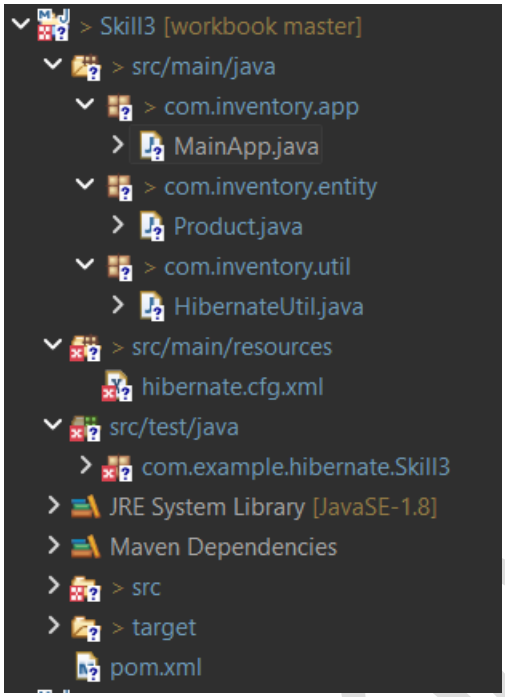
1. Use the Product entity created in Skill 2.
2. Add 5–8 additional Product records into the database.
3. Write HQL queries to retrieve all products sorted by price: a. Ascending order b. Descending order
4. Write an HQL query to sort products by quantity (highest first).
5. Implement pagination using HQL to display: a. First 3 products b. Next 3 products
6. Write HQL queries for aggregate operations: a. Count total number of products b. Count products where quantity > 0 c. Count products grouped by description d. Find minimum and maximum price
7. Write an HQL query using GROUP BY to group products by description.
8. Write an HQL query using WHERE to filter products within a price range.

2400030211

9. Write HQL queries using LIKE such as: a. Names starting with certain letters b. Names ending with certain letters c. Names containing a pattern anywhere (substring) d. Names with an exact character length

10. Push the Hibernate project to GitHub.

STRUCTURE:



CODES:

MainApp.java

```
package com.inventory.app;

import java.util.List;

import org.hibernate.Session;

import org.hibernate.Transaction;

import org.hibernate.query.Query;

import com.inventory.entity.Product;

import com.inventory.util.HibernateUtil;


public class MainApp {
```

240003021 1

```
public static void main(String[] args) {
    insertProducts();
    sortByPriceAsc();
    sortByPriceDesc();
    sortByQuantityDesc();
    pagination();
    aggregateQueries();
    groupByDescription();
    filterByPriceRange();
    likeQueries();
    HibernateUtil.getSessionFactory().close();
}
// Insert 5–8 products
static void insertProducts() {
    Session session = HibernateUtil.getSessionFactory().openSession();
    Transaction tx = session.beginTransaction();
    session.save(new Product("Laptop", "Electronics", 70000, 5));
    session.save(new Product("Mobile", "Electronics", 30000, 10));
    session.save(new Product("Tablet", "Electronics", 25000, 7));
    session.save(new Product("Mouse", "Accessories", 800, 50));
    session.save(new Product("Keyboard", "Accessories", 1500, 30));
    session.save(new Product("Monitor", "Electronics", 12000, 12));
    session.save(new Product("Printer", "Electronics", 18000, 4));
    tx.commit();
    session.close();
}
// Sort by price ascending
static void sortByPriceAsc() {
```

2400030211

```
    Session session = HibernateUtil.getSessionFactory().openSession();
    Query<Product> q = session.createQuery(
            "from Product order by price asc", Product.class);
    q.list().forEach(p ->
            System.out.println(p.getName() + " : " + p.getPrice()));
    session.close();
}
// Sort by price descending
static void sortByPriceDesc() {
    Session session = HibernateUtil.getSessionFactory().openSession();
    Query<Product> q = session.createQuery(
            "from Product order by price desc", Product.class);
    q.list().forEach(p ->
            System.out.println(p.getName() + " : " + p.getPrice()));
    session.close();
}
// Sort by quantity (highest first)
static void sortByQuantityDesc() {
    Session session = HibernateUtil.getSessionFactory().openSession();
    Query<Product> q = session.createQuery(
            "from Product order by quantity desc", Product.class);
    q.list().forEach(p ->
            System.out.println(p.getName() + " : " + p.getQuantity()));
    session.close();
}

// Pagination
static void pagination() {
```

240030211

```
    Session session = HibernateUtil.getSessionFactory().openSession();
    Query<Product> q = session.createQuery("from Product", Product.class);
    q.setFirstResult(0);
    q.setMaxResults(3);
    System.out.println("First 3 Products:");
    q.list().forEach(p -> System.out.println(p.getName()));
    q.setFirstResult(3);
    q.setMaxResults(3);
    System.out.println("Next 3 Products:");
    q.list().forEach(p -> System.out.println(p.getName()));
    session.close();
}
// Aggregate queries
static void aggregateQueries() {
    Session session = HibernateUtil.getSessionFactory().openSession();
    Long total = session.createQuery(
            "select count(*) from Product", Long.class).uniqueResult();
    Long available = session.createQuery(
            "select count(*) from Product where quantity > 0",
            Long.class).uniqueResult();
    Object[] minMax = session.createQuery(
            "select min(price), max(price) from Product",
            Object[].class).uniqueResult();
    System.out.println("Total Products: " + total);
    System.out.println("Available Products: " + available);
    System.out.println("Min Price: " + minMax[0]);
    System.out.println("Max Price: " + minMax[1]);
    session.close();
```

240030211

```
}
// Group by description
static void groupByDescription() {
    Session session = HibernateUtil.getSessionFactory().openSession();
    List<Object[]> list = session.createQuery(
            "select description, count(*) from Product group by description",
            Object[].class).list();
    for (Object[] obj : list) {
        System.out.println(obj[0] + " : " + obj[1]);
    }
    session.close();
}
// Filter by price range
static void filterByPriceRange() {
    Session session = HibernateUtil.getSessionFactory().openSession();
    Query<Product> q = session.createQuery(
            "from Product where price between 10000 and 40000",
            Product.class);
    q.list().forEach(p ->
            System.out.println(p.getName() + " : " + p.getPrice()));
    session.close();
}
// LIKE queries
static void likeQueries() {
    Session session = HibernateUtil.getSessionFactory().openSession();

    System.out.println("Starts with M:");
    session.createQuery(
```

240030211

```
            "from Product where name like 'M%'", Product.class)
            .list().forEach(p -> System.out.println(p.getName()));
        System.out.println("Ends with r:");
        session.createQuery(
            "from Product where name like '%r'", Product.class)
            .list().forEach(p -> System.out.println(p.getName()));
        System.out.println("Contains 'top':");
        session.createQuery(
            "from Product where name like '%top%'", Product.class)
            .list().forEach(p -> System.out.println(p.getName()));
        System.out.println("Exact length = 5:");
        session.createQuery(
            "from Product where length(name) = 5", Product.class)
            .list().forEach(p -> System.out.println(p.getName()));
        session.close();
    }
}
```

Product.java

```
package com.inventory.entity;
import javax.persistence.*;
@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String name;
```

240003021

```
    private String description;
    private double price;
    private int quantity;
    public Product() {}
    public Product(String name, String description, double price, int quantity) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.quantity = quantity;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public String getDescription() { return description; }
    public double getPrice() { return price; }
    public int getQuantity() { return quantity; }
    public void setPrice(double price) { this.price = price; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
}
```

HibernateUtil.java

```
package com.inventory.util;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateUtil {
    private static SessionFactory sessionFactory;
    static {
        try {
            sessionFactory = new Configuration()
                    .configure("hibernate.cfg.xml")
```

240030211

```
            .buildSessionFactory();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

hibernate.cfg.xml

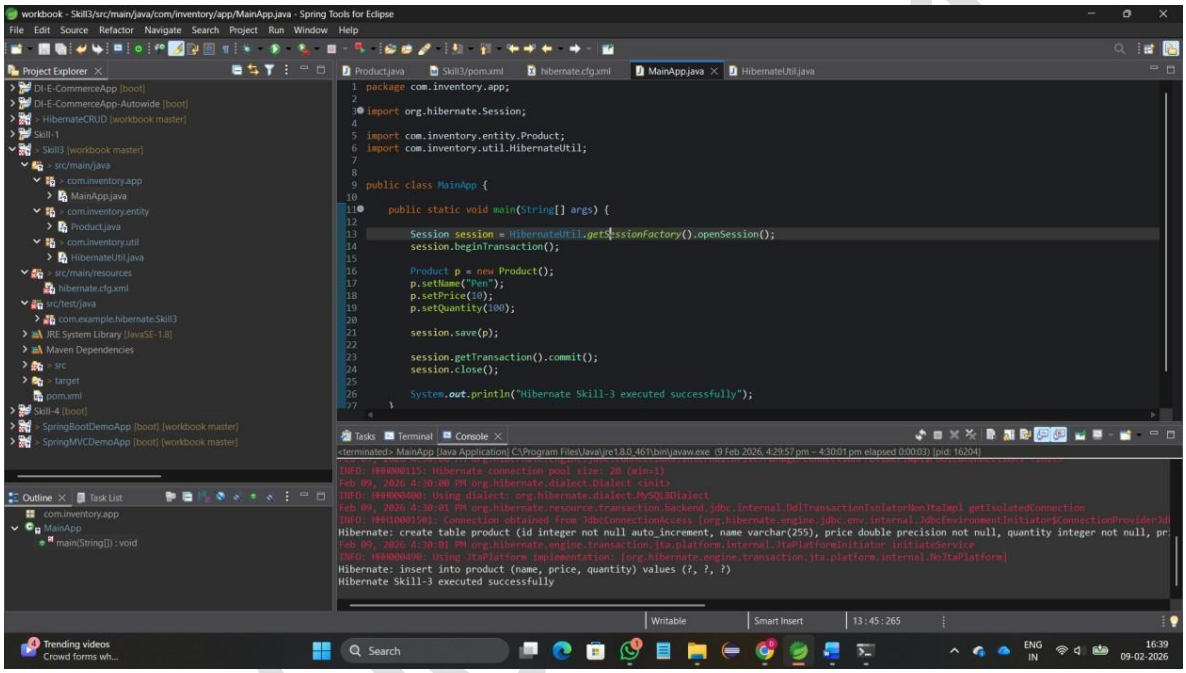
```
<?xml version="1.0" encoding="UTF-8"?>
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">
            com.mysql.cj.jdbc.Driver
        </property>
        <property name="hibernate.connection.url">
            jdbc:mysql://localhost:3306/skill3
        </property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">2006</property>
        <property name="hibernate.dialect">
            org.hibernate.dialect.MySQL8Dialect
        </property>

        <property name="hibernate.hbm2ddl.auto">update</property>
        <property name="hibernate.show_sql">true</property>
```

24003021 1

```
        <mapping class="com.inventory.entity.Product"/>
    </session-factory>
</hibernate-configuration>
```

OUTPUTS:



```
mysql> SHOW TABLES;
ERROR 1046 (3D000): No database selected
mysql> USE skill3_db;
Database changed
mysql> SHOW TABLES;
+---------------------+
| Tables_in_skill3_db |
+---------------------+
| product             |
+---------------------+
1 row in set (0.01 sec)

mysql> SELECT * FROM product;
+----+------+-------+----------+
| id | name | price | quantity |
+----+------+-------+----------+
|  1 | Pen  |    10 |      100 |
+----+------+-------+----------+
1 row in set (0.00 sec)

mysql> |
```

Git link:
https://github.com/hemalatharachakonda/2400030211-Skill_experiments.git

Viva Questions:

1. What is HQL and how does it differ from SQL?
HQL (Hibernate Query Language) is an object-oriented query language used to query Hibernate entities.
Unlike SQL (which works on tables and columns), HQL works on classes and their properties and is database-independent.

---

2. Why do we use sorting in HQL?
Sorting in HQL is used to arrange query results in ascending or descending order using the ORDER BY clause.

---

3. What is pagination and where is it used?
Pagination is the process of dividing large result sets into smaller pages.

2400030211

It is used in applications to improve performance and display data page-by-page (e.g., search results, records list).

---

4. Name any three aggregate functions supported in HQL.

- count()
- sum()
- avg()

---

5. What is the difference between HQL and HCQL (Hibernate Criteria Query Language)?

- HQL: String-based, written like SQL queries.
- HCQL (Criteria): Object-oriented, programmatic query creation without writing SQL-like strings.

240003021